A2.1 — Chủ đề, Paper và Dữ liệu

Bài tập lớn A2: Phát hiện đối tượng — CNN (Faster R-CNN) vs Transformer (DETR)

- **Môn:** Học sâu và ứng dụng trong thị giác máy tính (CO5085) — HK261 (2026–2027)
- **Giảng viên:** Lê Thành Sách, Nguyễn Quốc Minh
- **Nhóm:** Hummingbird
- **Thành viên:** Nguyễn Lê Hoàng Hưng (2570214), Nguyễn Thị Linh Trang (2570344)
- **Ngày nộp:** 02/11/2026

---

1. Chủ đề

Phát hiện đối tượng (object detection), chủ đề số 2 trong đề: dự đoán bounding box kèm nhãn lớp cho mọi đối tượng trong ảnh.

Nhóm so sánh hai cách tiếp cận khác nhau về bản chất:

	Faster R-CNN (CNN)	**DETR (Transformer)**
Cách sinh dự đoán	Anchor box dày đặc trên feature map, qua RPN rồi phân loại từng vùng	100 *object query* cố định, decoder Transformer dự đoán song song
Gán dự đoán với ground truth	Theo IoU với anchor (luật thủ công)	Ghép cặp một-một bằng thuật toán Hungarian
Loại bỏ box trùng	Cần NMS	Không cần NMS
Ngữ cảnh	Cục bộ (receptive field của conv)	Toàn cục (self-attention trên toàn ảnh)

Câu hỏi chính: trên một tập dữ liệu nhỏ hơn COCO nhiều và với một GPU phổ thông, DETR có giữ được những ưu và nhược điểm mà paper báo cáo hay không.

---

2. Paper được phân tích

> N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko. **End-to-End Object Detection with Transformers.** *European Conference on Computer Vision (ECCV)*, 2020. arXiv: 2005.12872. Code: github.com/facebookresearch/detr (Apache-2.0).

Lý do chọn:

1. Liên quan trực tiếp tới chủ đề: DETR là mô hình phát hiện đối tượng đầu tiên dùng Transformer end-to-end, bỏ anchor và NMS.
2. Paper có so sánh trực tiếp với Faster R-CNN, đúng cặp phương pháp nhóm chọn. Nhờ vậy có thể đối chiếu kết quả của nhóm với kết quả của paper.

3. Có code và checkpoint chính thức, nên phần cài đặt bám sát được phương pháp gốc.
4. Nối tiếp với A1 (CNN vs Transformer trong phân loại ảnh): A2 mở rộng câu hỏi này sang bài toán phát hiện.

Các kết quả của paper mà nhóm sẽ kiểm chứng (COCO val, ResNet-50, Bảng 1 và Bảng 2 của paper):

Kết quả trong paper	Số liệu	Nhóm kiểm chứng bằng
DETR ngang Faster R-CNN về AP tổng	DETR 42,0 AP; Faster R-CNN-FPN+ 42,0 AP	So sánh mAP trên VOC 2007 test
DETR kém hơn ở vật nhỏ	AP_S 20,5 so với 26,6	AP_S của hai mô hình
DETR tốt hơn ở vật lớn	AP_L 61,1 so với 53,4	AP_L của hai mô hình
DETR cần lịch train rất dài	300 epoch trên 16 GPU V100 mất 3 ngày; bản tốt nhất 500 epoch	Train DETR chỉ với backbone ImageNet, phần Transformer khởi tạo ngẫu nhiên (biến thể V3)
Encoder quan trọng nhờ suy luận toàn cục	Bỏ encoder: AP 40,6 xuống 36,7	Tùy chọn, nếu đủ thời gian GPU
Cùng cỡ mô hình và tốc độ	DETR 41M tham số, 28 FPS; Faster R-CNN-FPN 42M tham số, 26 FPS	Đếm tham số, đo FPS trên GTX 1070

3. Dữ liệu

3.1 Pascal VOC 2007

Thuộc tính	Giá trị
Nguồn	M. Everingham et al., *The PASCAL Visual Object Classes (VOC) Challenge*, IJCV 2010
Truy cập	`torchvision.datasets.VOCDetection(year="2007")`; dự phòng bản mirror trên Kaggle/Hugging Face nếu server gốc không truy cập được
Số lớp	20 (người, động vật, phương tiện, đồ vật trong nhà)
trainval	5.011 ảnh, 12.608 đối tượng
test	4.952 ảnh, 12.032 đối tượng, có nhãn đầy đủ
Nhãn	Bounding box và lớp; cờ `difficult` cho vật khó nhận
License	Ảnh lấy từ Flickr, chỉ dùng cho nghiên cứu và học tập. Nhóm không phân phối lại ảnh; repo chỉ chứa code tải dữ liệu.

3.2 Lý do chọn VOC 2007 thay vì COCO

1. **Vừa sức tính toán:** COCO có 118 nghìn ảnh train; một lần train DETR trên COCO cần nhiều ngày với 16 GPU. VOC 2007 nhỏ hơn khoảng 24 lần, nên fine-tune trên một GTX 1070 là khả thi.
2. **Đủ lớn để đánh giá có ý nghĩa:** gần 5.000 ảnh test với hơn 12.000 đối tượng. VOC là benchmark chuẩn của bài toán phát hiện.

3. **Có metric chuẩn:** mAP@0.5 là metric chính thức của VOC. Ngoài ra nhóm tính thêm metric kiểu COCO để so với paper.
4. **Nhiều kích thước vật thể:** có cả vật nhỏ (chai, chim) lẫn vật lớn (tàu hỏa, ghế sofa), nên kiểm chứng được nhận định về AP_S và AP_L.

Lưu ý về pretrained: cả 20 lớp của VOC đều có trong 80 lớp của COCO. Fine-tune từ checkpoint COCO vì thế là học chuyển giao (transfer learning) giữa hai tập dữ liệu, không phải train lại từ đầu. Ảnh của VOC 2007 không nằm trong COCO, nên tập test không bị rò dữ liệu. Biến thể V3 (chỉ backbone ImageNet) được thêm vào để có một cấu hình không dùng kiến thức phát hiện học từ COCO.

3.3 Split

Split	Cách lấy	Số ảnh
Train	90% VOC 2007 trainval, chia ngẫu nhiên với seed 42	~4.510
Validation	10% còn lại của trainval	~501
Test	Toàn bộ VOC 2007 test	4.952

- Validation dùng để chọn checkpoint và hyperparameter. Test chỉ dùng **một lần** cho kết quả cuối.
- Danh sách ảnh của từng split được lưu vào `A2/splits/` và commit lên repo.
- Đối tượng có cờ `difficult` được giữ khi train và bỏ qua khi tính mAP, theo đúng quy ước của VOC.

3.4 EDA dự kiến (trình bày ở A2.2)

- Số đối tượng mỗi lớp (VOC lệch lớp mạnh: lớp `person` chiếm phần lớn).
- Số đối tượng mỗi ảnh, kích thước ảnh.
- Phân bố diện tích bounding box theo ngưỡng nhỏ/vừa/lớn của COCO ($< 32^2$, 32^2–96^2, $> 96^2$ pixel).
- Ảnh mẫu kèm box ground truth.

---

4. Metric

Metric	Ý nghĩa	Vai trò
mAP@0.5	Trung bình AP của 20 lớp, box đúng khi IoU ≥ 0,5	Metric chính (chuẩn VOC)
mAP@0.5:0.95	Trung bình mAP ở 10 ngưỡng IoU từ 0,5 đến 0,95	Đánh giá độ chính xác vị trí box; so được với paper
AP_S, AP_M, AP_L	mAP@0.5:0.95 theo kích thước vật thể	Kiểm chứng nhận định về vật nhỏ và vật lớn
AP theo từng lớp	AP@0.5 của mỗi lớp	Phân tích lỗi
FPS	Số ảnh suy luận mỗi giây trên GTX 1070, batch 1	Chi phí
Tham số, thời gian train, bộ nhớ GPU		Chi phí

Công cụ: `pycocotools` (chuyển nhãn VOC sang định dạng COCO) hoặc `torchmetrics.detection.MeanAveragePrecision`. Hai công cụ này cho cùng định nghĩa AP kiểu COCO.

---

5. Kế hoạch pipeline

5.1 Các phương pháp và biến thể

#	Mô hình	Khởi tạo	Mục đích
V1	Faster R-CNN, ResNet-50-FPN (`torchvision`)	Checkpoint COCO	Baseline CNN: anchor + NMS
V2	DETR, ResNet-50 (`detr-main`)	Checkpoint COCO chính thức	Transformer: set prediction, không NMS
V3	DETR, ResNet-50	Chỉ backbone ImageNet; Transformer khởi tạo ngẫu nhiên	Kiểm chứng việc DETR hội tụ chậm
V4 *(nếu đủ thời gian)*	DETR, ResNet-50	Checkpoint COCO, tắt auxiliary decoding loss	Ablation theo paper
V5 *(nếu đủ thời gian)*	Faster R-CNN, ResNet-50-FPN	Chỉ backbone ImageNet	Đối xứng với V3: so sánh hai họ khi không có pretrain phát hiện

So sánh chính theo yêu cầu của đề (≥ 2 phương pháp): **V1 so với V2** (CNN so với Transformer) và **V2 so với V3** (có và không có pretrain phát hiện).

5.2 Cài đặt

- **Faster R-CNN:** `torchvision.models.detection.fasterrcnn_resnet50_fpn(weights="COCO_V1")`, thay `box_predictor` bằng `FastRCNNPredictor(1024, 21)` (20 lớp + nền).

- **DETR:** dùng các module của repo `detr-main` (`models/backbone.py`, `transformer.py`, `matcher.py`, `detr.py` với `SetCriterion`). Nạp checkpoint `detr-r50-e632da11.pth`, thay `class_embed` bằng `Linear(256, 21)` (20 lớp + "không có vật thể"), giữ 100 object query.
- **Loss của DETR theo paper:** cross-entropy cho lớp, L1 và GIoU cho box (trọng số 1 / 5 / 2), ghép cặp Hungarian, auxiliary loss sau mỗi tầng decoder.
- **Training loop tự viết,** dùng chung cho cả hai mô hình: forward → loss → `backward()` → `optimizer.step()`; data loader và hàm đánh giá dùng chung để so sánh công bằng.

5.3 Hyperparameter dự kiến

	Faster R-CNN (V1, V5)	**DETR (V2, V3, V4)**
Optimizer	SGD, momentum 0,9, weight decay 1e-4	AdamW, weight decay 1e-4 (theo paper)
Learning rate	0,01, giảm 10 lần ở 2/3 lịch train	Transformer 1e-4, backbone 1e-5 (theo paper)
Gradient clipping	—	max norm 0,1 (theo paper)
Epoch	12	25, giảm learning rate ở epoch 20
Batch	4	4

5.4 Augmentation (giống nhau cho mọi mô hình)

- Lật ngang ngẫu nhiên.
- Multi-scale resize: cạnh ngắn chọn ngẫu nhiên trong {384, 416, …, 608}, cạnh dài tối đa 1000 pixel. Paper dùng 480–800, nhóm giảm xuống để vừa 8GB VRAM.
- Random crop với xác suất 0,5 như paper (chỉ khi còn thời gian GPU; ghi rõ trong báo cáo nếu bỏ).
- Đánh giá: cạnh ngắn 512 pixel.

5.5 Phần cứng và rủi ro

- **Phần cứng:** GPU NVIDIA GeForce GTX 1070 (8GB), PyTorch 2.8 bản CUDA 12.6. Dự phòng: GPU T4 trên Kaggle.
- **Rủi ro 1, thời gian GPU:** ước tính 3–6 giờ mỗi run (sẽ đo lại thực tế). A1.2 (hạn 09/11) cũng cần GPU, nên các run chính của A2 xếp sau ngày 09/11. Nếu không đủ thời gian, V4 và V5 sẽ bỏ, và số epoch của V3 sẽ được ghi rõ trong báo cáo.
- **Rủi ro 2, repo cũ:** `detr-main` đã ngừng phát triển từ 2024. Nhóm sẽ chạy thử trên PyTorch 2.8 trước bằng cách overfit vài ảnh. Đoạn kiểm tra phiên bản torchvision trong `util/misc.py` dùng `packaging.version`, nên không gặp lỗi so sánh chuỗi phiên bản.
- **Rủi ro 3, bộ nhớ:** nếu batch 4 không vừa 8GB thì dùng batch 2 với gradient accumulation 2 bước.

---

6. Kế hoạch phân tích paper (A2.2)

1. **Phương pháp:** bài toán phát hiện dưới dạng dự đoán tập hợp; ghép cặp Hungarian và loss; kiến trúc backbone, encoder, decoder, object query; positional encoding; auxiliary loss. Viết lại bằng lời của nhóm, gắn với code `detr-main`.

2. **Thí nghiệm của paper:** COCO, các metric AP; so sánh với Faster R-CNN; ablation số tầng encoder và decoder, các thành phần của loss; mở rộng sang panoptic segmentation.
3. **Hạn chế:** chi phí train rất lớn; yếu ở vật nhỏ do chỉ dùng một feature map độ phân giải thấp (stride 32); số object query cố định (100); những giả thiết của paper không còn đúng trên VOC và GTX 1070.
4. **Đối chiếu với nhóm:** kết quả nào tái hiện được trên VOC, kết quả nào không, vì sao, và điều đó gợi ý gì cho pipeline. Ví dụ: Deformable DETR được đề xuất sau đó để sửa đúng hai điểm yếu là hội tụ chậm và vật nhỏ.

---

7. Lịch thực hiện

Thời gian	Công việc
02/11/2026	**Nộp A2.1:** chủ đề, paper, dữ liệu, metric, kế hoạch
03/11 – 08/11	Tải VOC, tạo split, EDA; chuyển nhãn sang COCO; chạy thử hai pipeline bằng cách overfit vài ảnh; kiểm tra hàm đánh giá trên checkpoint COCO
10/11 – 14/11	Chạy V1, V2, V3 trên GTX 1070 (sau hạn A1.2); V4, V5 nếu đủ thời gian
14/11 – 16/11	Tổng hợp kết quả, phân tích lỗi, viết phân tích paper và báo cáo PDF; cập nhật GitHub Pages. **Nộp A2.2**